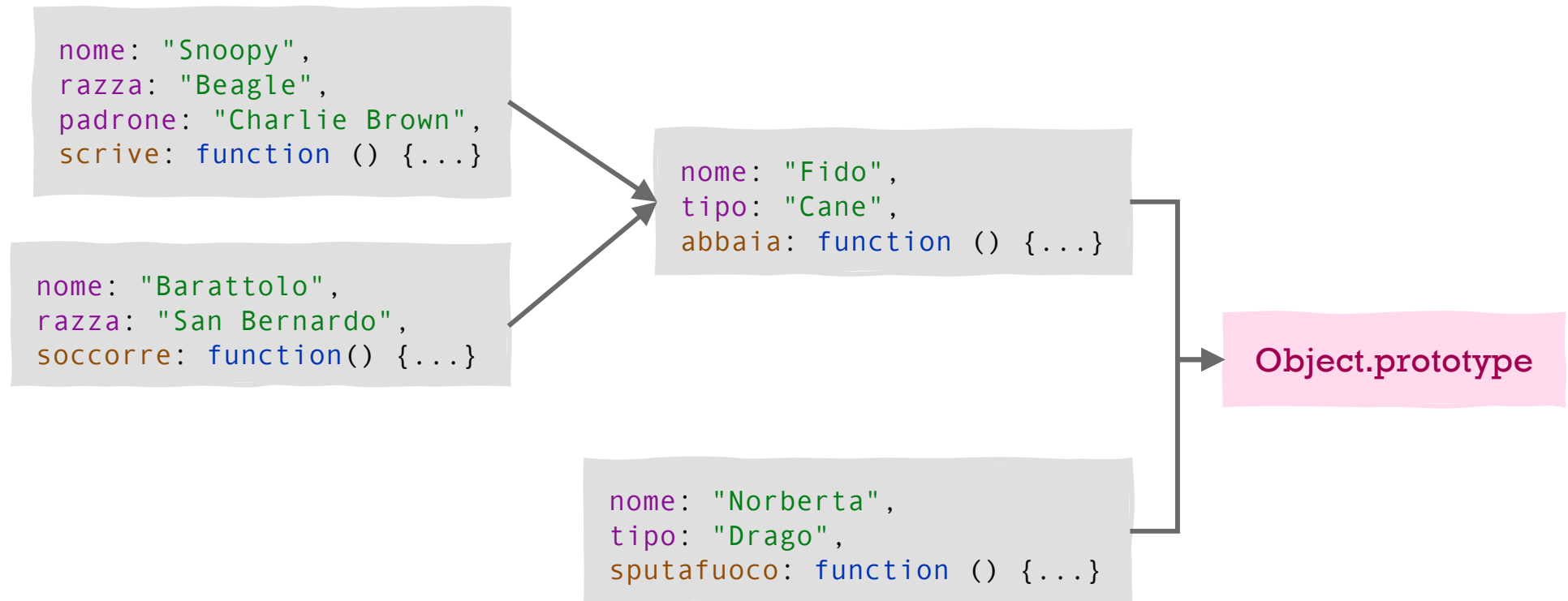


Ereditarietà in Javascript

I prototipi

In Javascript ogni* oggetto ha un **prototipo**: un altro oggetto di cui eredita le proprietà.

(*) L'unico oggetto privo di prototipo è l'oggetto **Object.prototype**, che è il prototipo di default degli object literal



Oggetti e prototipi - I

In Javascript esistono due **oggetti primordiali** che formano la base di tutte le strutture dati complesse del linguaggio:

- La **funzione Object**. Si tratta della funzione che viene **implicitamente** invocata ogni qualvolta c'è da creare un oggetto (ad esempio, quando dichiariamo un object literal)
- L'**oggetto Object.prototype**, che fa da **prototipo di default** per tutti gli oggetti creati senza specificare un prototipo diverso

Se la scrittura Object.prototype vi suona strana, avendo appena detto che Object è una funzione, pensate al fatto che in Javascript una funzione è un oggetto, per quanto un oggetto particolare, e può dunque possedere delle proprietà al pari degli altri oggetti.

Cosa significa che un oggetto è prototipo di un altro? Significa che quando cerchiamo di **leggere** una proprietà di un oggetto (per conoscerne il valore, o, se si tratta di una proprietà-funzione, per invocarla) il **motore di esecuzione**, se non trova la proprietà nell'oggetto stesso, andrà a cercarla nel suo prototipo.

Per questo si dice che l'oggetto **eredita** le proprietà del prototipo.

Il prototipo viene invece ignorato quando **assegniamo un valore** a una proprietà di un oggetto: se la proprietà non esiste nell'oggetto stesso, gli viene aggiunta. Se il prototipo aveva una proprietà con lo stesso modo, siamo in presenza del meccanismo di **overriding**: un erede può sovrascrivere una proprietà del proprio antenato. Il motore di esecuzione troverà prima la proprietà «locale» e ignorerà quella ereditata con lo stesso nome.

Se anche un erede sovrascrive per sé stesso una proprietà, gli altri eredi possono continuare a condividere quella ereditaria con il proprio prototipo.

Oggetti e prototipi - II

Il fatto che ogni oggetto abbia un prototipo, il quale è a sua volta un oggetto con un prototipo, e così via, fino ad arrivare all'unico oggetto privo di prototipo, forma la cosiddetta **prototype chain** o **catena dei prototipi**.

Prima di decidere che un oggetto non **possiede** una certa proprietà, il motore di esecuzione risale tutta la catena dei prototipi fino ad arrivare a **Object.prototype**.

Possiamo accedere in ogni momento al prototipo di un dato oggetto tramite:

oggetto.__proto__

Tramite esso possiamo **modificare** le proprietà del prototipo, di fatto modificando le proprietà accessibili a **tutti gli oggetti** che fanno capo a quel prototipo.

Se ad esempio eliminiamo la proprietà-funzione *abbaia* dall'oggetto **fido**, anche gli oggetti **snoopy** e **cucciolo** la perderanno.

Questo vale anche per i prototipi predefiniti. Se aggiungiamo una proprietà a **Object.prototype**, improvvisamente **tutti gli oggetti** della nostra applicazione la acquisiscono! In pratica possiamo in qualsiasi momento... distruggere o plasmare il linguaggio a nostro piacimento... e a nostro rischio e pericolo!

È possibile (ma non è una funzionalità molto usata) «bloccare» le possibilità di modificare un oggetto dopo la sua creazione tramite:

Object.preventExtensions(ogg) → **ogg** non potrà acquisire nuove proprietà o cambiare prototipo

Object.seal(ogg) → *sigilla* **ogg**, che non potrà acquisire nuove proprietà o cambiare prototipo, ma non potrà nemmeno **perdere** proprietà.

Object.freeze(ogg) → *congela* **ogg**: oltre a *sigillarlo*, impedisce anche che nuovi valori vengano assegnati alle sue proprietà, rendendolo di fatto **read only**

Ereditarietà in Javascript

Creazione di oggetti a partire da un prototipo

```
const fido = {  
  tipo: "Cane",  
  nome: "Fido",  
  abbaia: function () {}  
}
```

Creazione di un object literal con prototipo di default, ossia **Object.prototype**

```
const snoopy = Object.create(fido);  
snoopy.nome = "Snoopy";  
snoopy.razza = "Beagle";  
snoopy.padrone = "Charlie Brown";  
snoopy.scrive = function () {};
```

Creazione di un object literal specificandone il prototipo (nell'esempio l'oggetto **fido**). Successivamente vengono assegnate proprietà al nuovo oggetto, alcune sono «originali», altre sono un «overriding» delle proprietà dell'oggetto base.

```
const cucciolo = Object.create(fido);  
cucciolo.razza = "Jack Russel Terrier";  
cucciolo.scodinzola = function () {};
```

Questo terzo oggetto, finché non ridefinisce la proprietà **nome**, eredita la proprietà **nome** del suo prototipo, il cui valore è "Fido"

Ereditarietà in Javascript

Determinare le proprietà di un oggetto

```
Object.keys(snoopy);  
/* ['nome', 'razza',  
    'padrone', 'scrive'] */
```

Restituisce un **array** contenente i **nomi** delle proprietà (chiavi o **key**) definite nell'oggetto stesso

```
const allprops = [];  
for (let key in snoopy)  
    allprops.push(key);  
  
console.log(allprops);  
/* ['nome', 'razza', 'padrone',  
    'scrive', 'tipo', 'abbaia'] */
```

for..in cicla su tutte le chiavi valide per l'oggetto, **include** quelle della prototype chain, **ma escluse** quelle «di sistema» (in questa categoria sono ad esempio incluse tutte le proprietà definite in `Object.prototype`, che quindi non vengono elencate)

```
snoopy.hasOwnProperty("nome");  
/* true */  
snoopy.hasOwnProperty("tipo");  
/* false */
```

Il metodo **hasOwnProperty**, definito in **Object.prototype**, e dunque disponibile a tutti gli oggetti, determina se un oggetto definisce una proprietà; se la proprietà non esiste o è ereditaria, restituisce **false**.

Metodi di un oggetto in Javascript

Il binding con **this**

Un metodo di un oggetto è una **funzione** che viene collegata (**bound**) ad un certo oggetto, che diventerà il **soggetto** della funzione stessa

Nell'esecuzione della funzione la parola chiave **this** si riferirà a tale oggetto-soggetto.

```
const fido = {  
  tipo: "Cane",  
  nome: "Fido",  
  abbaia: function () {} /* metodo */  
  corre: () => {} /* non metodo */  
}
```

Il **binding** avviene implicitamente per le **proprietà-funzione** di un oggetto che sono definite come **named functions** o **anonymous functions**.

Non avviene implicitamente per le proprietà-funzione definite come **arrow functions**.

```
/* binding esplicito */  
function log(msg) {  
  console.log(msg + ": " + this.nome);  
}  
const stampaFido = log.bind(fido);  
  
/* binding implicito */  
fido.stampa = log;
```

Il metodo **f.bind** (dove **f** è una funzione) crea una copia della funzione che è **bound** all'oggetto specificato.

Si ottiene il medesimo effetto assegnando la funzione ad una proprietà dell'oggetto **a meno che non fosse una arrow function**, nel qual caso il binding potrà solo essere **esplicito**.

I costruttori in Javascript

Creare oggetti con lo stampino

Il sistema originario in Javascript per creare oggetti «con lo stampino» è quello di definire dei **costruttori**

I costruttori sono **named functions** (per convenzione, i loro nomi iniziano tipicamente con la maiuscola) che vengono invocate premettendo la keyword **new**

```
function Cane(nome, razza) {  
    this.tipo = "Cane";  
    this.nome = nome;  
    if (razza !== undefined)  
        this.razza = razza;  
}  
  
const fido = new Cane("Fido");  
const snoopy = new Cane("Snoopy", "Beagle");
```

Quando chiamiamo una funzione **costruttore Fun** con **new** succede quanto segue:

- viene creato un oggetto da usare come prototipo, sarà memorizzato in **Fun.prototype**
- viene creato un nuovo oggetto **x** con prototipo **Fun.prototype**
- viene realizzato il **binding** fra il nuovo oggetto e la funzione: dunque **this** all'interno della funzione si riferisce al nuovo oggetto **x** appena creato

Dunque, tutti gli oggetti creati tramite **new Fun(...)** condividono il medesimo prototipo©

Costruttori e Classi

Classe come condivisione del prototipo

Il concetto di **costruttore** è alla base del concetto di **classe** in Javascript

Due oggetti sono della stessa **classe** se sono creati a partire dal medesimo costruttore.

```
function Cane(nome, razza) {  
  this.tipo = "Cane";  
  this.nome = nome;  
  if (razza !== undefined)  
    this.razza = razza;  
}  
  
Cane.prototype.abbaia = function() {}  
  
const fido = new Cane("Fido");  
fido.abbaia();
```

Due oggetti creati dallo stesso costruttore vengono inizializzati nello stesso modo, quindi (almeno in partenza) hanno le stesse proprietà (ognuno con un proprio valore, la proprietà è dell'oggetto, non del prototipo!)

Poiché gli oggetti hanno anche lo stesso prototipo, è possibile assegnare **metodi** alla «classe» semplicemente aggiungendoli al prototipo.

In effetti questo è effettivamente ciò che si «nasconde» dietro al costrutto **class** in Javascript. La «classe» che descriviamo non è altro che un **costruttore** e i metodi che inseriamo «dentro» la classe vengono assegnati al **prototipo** associato al costruttore

Costruttori e Classi

Classe come condivisione del prototipo

```
function Cane(nome, razza) {  
  this.tipo = "Cane";  
  this.nome = nome;  
  if (razza !== undefined)  
    this.razza = razza;  
}
```

```
Cane.prototype.abbaia = function() {}
```

```
const fido = new Cane("Fido");  
fido.abbaia();
```

```
class Cane {
```

```
  constructor(nome, razza) {  
    this.tipo = "Cane";  
    this.nome = nome;  
    if (razza !== undefined)  
      this.razza = razza;  
  }
```

```
  abbaia() {}  
}
```

```
const fido = new Cane("Fido");  
fido.abbaia();
```

Ereditarietà fra classi

Con costruttori e prototipi

```
class Cane {  
    constructor(nome, razza) {  
        ...  
    }  
    abbaia() {}  
}  
  
const fido = new Cane("Fido");
```

```
class Beagle extends Cane {  
    constructor(nome) {  
        super("Beagle");  
        ...  
    }  
    scrive() {}  
}  
  
const snoopy = new Beagle("Snoopy");
```

